Key milestones of Java releases

Year	Version	Milestone
1995	Java 1.0	Initial release
1999	Java 2 (J2SE 1.2)	Introduction of Swing API
2004	Java 5	Enhanced for Loop, Generics
2011	Java 7	Project Coin, NIO.2 file system
2014	Java 8	Lambda expressions, Stream API
2017	Java 9	Module system (Project Jigsaw)
2018	Java 10	Local-variable type inference
2019	Java 12	Switch expressions (Preview)
2021	Java 16	Records, Sealed classes
2023	Java 20	Pattern matching for switch expressions
2025	Java 24	Introduction of advanced features (detailed below)

Java 24 introduces several noteworthy features aimed at enhancing performance, memory handling, and developer productivity. Here is an overview of these features with corresponding examples.

Garbage collection enhancements

Java 24 brings further optimisations to garbage collection. The introduction of the new G1GC (Garbage First Garbage Collector) improves memory management and reduces pause times significantly. Here is an example demonstrating the optimisation:

```
public class GarbageCollectionExample {
public static void main(String[] args) {
for (int i = 0; i < 10000; i++) {
String temp = "String " + i;
}
System.out.println("Garbage Collection optimization in
action.");
}
}
```

Memory handling improvements

Memory handling has been fine-tuned in Java 24. The introduction of new APIs helps in better management and monitoring of memory usage. This ensures that applications run smoothly without unnecessary memory overhead.

```
public class MemoryHandlingExample {
public static void main(String[] args) {
long allocatedMemory = Runtime.getRuntime().totalMemory() -
Runtime.getRuntime().freeMemory();
System.out.println("Allocated Memory: " + allocatedMemory + "
bytes");
}
}
```

Performance enhancements

Java 24 focuses on performance with features like enhanced JIT (just-in-time) compilation and improvements in the HotSpot virtual machine. These features ensure faster execution of Java applications.

Lambda expressions

Lambda expressions, a feature introduced in Java 8, receive further enhancements in Java 24, making them even more powerful and versatile.

```
import java.util.stream.IntStream;
public class LambdaExample {
public static void main(String[] args) {
IntStream.range(1, 10).filter(i -> i % 2 == 0).forEach(System.
out::println);
}
}
```

Integration packages

Java 24 introduces new integration packages that facilitate seamless integration with modern development frameworks and tools. This enhances the interoperability of Java with other technologies, streamlining the development process.

Structured concurrency

Structured concurrency in Java 24 is an incubator feature introduced to improve the maintainability, reliability, and observability of multi-threaded code. It enables tasks to be divided into smaller parts that can be executed in parallel in virtual threads within clearly defined source code blocks. This feature aims to minimise thread leaks and delays associated with cancellation and shutdown.

Structured concurrency capabilities are available in Java 19 and later versions, helping developers manage multithreaded code more effectively. To use structured concurrency in Java 24, follow these general steps**.**

Create a StructuredTaskScope*:* Use it with a 'try-with-resources' statement.

Define your subtasks*:* These should be instances of Callable.

Fork each subtask*:* Within the try block, fork each subtask in its own thread using *StructuredTaskScope::fork.*

Join the subtasks*:* Call *StructuredTaskScope::join* to wait for all subtasks to finish execution.

Handle the outcome*:* Process the results or exceptions from the subtasks.